

Backend Assignments — Complete Code, API Testing & Expected Outputs

Technologies: Node.js, Express.js, MongoDB, Mongoose, bcryptjs, JWT and Postman

Assignment 1 — To-Do List REST API

Purpose: Create and manage tasks using REST endpoints and MongoDB.

API endpoints

Method	Endpoint	Purpose
GET	/	Check API status
POST	/api/tasks	Add task
GET	/api/tasks	Get all tasks
PUT	/api/tasks/:id	Update task
DELETE	/api/tasks/:id	Delete task

Postman testing example

```
POST http://localhost:5000/api/tasks
Body → raw → JSON
{
  "title": "Complete backend assignment"
}

Expected:
{
  "_id": "TASK_ID",
  "title": "Complete backend assignment",
  "completed": false
}
```

Expected output / console flow

```
MongoDB connected
Server running on http://localhost:5000

POST /api/tasks → 201 Created
GET /api/tasks → 200 OK
PUT /api/tasks/:id → 200 OK
DELETE /api/tasks/:id → 200 OK

Example response:
{ "message": "Task deleted successfully" }
```

Source code

.env.example

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/todo_db
```

README.md

```
# Assignment 1 - To-Do List REST API

## Run
1. Install Node.js and MongoDB.
2. Copy `.env.example` to `.env`.
3. Run `npm install`.
4. Run `npm start`.
5. Test using Postman.

## Endpoints

| Method | Endpoint | Purpose |
|---|---|---|
| GET | / | Check API |
| POST | /api/tasks | Add task |
| GET | /api/tasks | Get all tasks |
| PUT | /api/tasks/:id | Update task |
| DELETE | /api/tasks/:id | Delete task |

## Postman Examples
POST http://localhost:5000/api/tasks
```

```

JSON body:
{
  "title": "Complete backend assignment"
}

PUT http://localhost:5000/api/tasks/TASK_ID
JSON body:
{
  "title": "Complete backend assignment",
  "completed": true
}

DELETE http://localhost:5000/api/tasks/TASK_ID

Expected successful add response:
{
  "_id": "TASK_ID",
  "title": "Complete backend assignment",
  "completed": false
}

```

package.json

```

{
  "name": "todo-rest-api",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.21.1",
    "mongoose": "^8.7.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.7"
  }
}

```

server.js

```

require("dotenv").config();

const express = require("express");
const mongoose = require("mongoose");
const taskRoutes = require("./routes/taskRoutes");

const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.json({ message: "To-Do List REST API is running" });
});

app.use("/api/tasks", taskRoutes);

const PORT = process.env.PORT || 5000;

mongoose
  .connect(process.env.MONGODB_URI)
  .then(() => {
    console.log("MongoDB connected");
    app.listen(PORT, () => {
      console.log(`Server running on http://localhost:${PORT}`);
    });
  })
  .catch((error) => {
    console.error("MongoDB connection error:", error.message);
  });

```

models/Task.js

```

const mongoose = require("mongoose");

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: true,
      trim: true
    },
    completed: {
      type: Boolean,
      default: false
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model("Task", taskSchema);

```

routes/taskRoutes.js

```

const express = require("express");
const Task = require("../models/Task");

```

```

const router = express.Router();

// Add task
router.post("/", async (req, res) => {
  try {
    const { title } = req.body;

    if (!title) {
      return res.status(400).json({ message: "Title is required" });
    }

    const task = await Task.create({ title });
    res.status(201).json(task);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Get all tasks
router.get("/", async (req, res) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 });
    res.json(tasks);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Update task
router.put("/:id", async (req, res) => {
  try {
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!task) {
      return res.status(404).json({ message: "Task not found" });
    }

    res.json(task);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Delete task
router.delete("/:id", async (req, res) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);

    if (!task) {
      return res.status(404).json({ message: "Task not found" });
    }

    res.json({ message: "Task deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;

```

Assignment 2 — User Authentication API

Purpose: Register/login users with bcrypt password hashing and JWT authentication.

API endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login and get JWT
GET	/api/auth/profile	Protected profile

Postman testing example

POST http://localhost:5001/api/auth/register
Body → raw → JSON

```
{
  "name": "Vedika",
  "email": "vedika@example.com",
  "password": "Password123"
}
```

Expected:

```
{
  "message": "User registered successfully",
  "user": {
    "id": "USER_ID",
    "name": "Vedika",
    "email": "vedika@example.com"
  }
}
```

Then POST /api/auth/login with the same email/password.

Expected:

```
{
  "message": "Login successful",
  "token": "JWT_TOKEN"
}
```

For GET /api/auth/profile:

Authorization → Bearer Token → paste JWT_TOKEN.

Expected output / console flow

MongoDB connected
Server running on http://localhost:5001

POST /api/auth/register → 201 Created
POST /api/auth/login → 200 OK
GET /api/auth/profile → 200 OK with Bearer JWT

Invalid/missing JWT → 401 Unauthorized

Source code

.env.example

```
PORT=5001
MONGODB_URI=mongodb://127.0.0.1:27017/auth_db
JWT_SECRET=change_this_to_a_long_random_secret
```

README.md

Assignment 2 - User Authentication API

Run

1. Copy `.env.example` to `.env`.
2. Set a private `JWT_SECRET`.
3. Run `npm install`.
4. Run `npm start`.

Endpoints

POST /api/auth/register
POST /api/auth/login
GET /api/auth/profile (protected)

Register JSON

```
{
  "name": "Vedika",
  "email": "vedika@example.com",
  "password": "Password123"
}
```

Login JSON

```
{
  "email": "vedika@example.com",
  "password": "Password123"
}
```

```
}
```

Copy the returned JWT token. For the profile request use:
Authorization: Bearer YOUR_TOKEN

Expected login response:

```
{
  "message": "Login successful",
  "token": "JWT_TOKEN"
}
```

Passwords are stored as bcrypt hashes, not plain text.

package.json

```
{
  "name": "user-authentication-api",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "dotenv": "^16.4.5",
    "express": "^4.21.1",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.7.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.7"
  }
}
```

server.js

```
require("dotenv").config();

const express = require("express");
const mongoose = require("mongoose");
const authRoutes = require("./routes/authRoutes");

const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.json({ message: "User Authentication API is running" });
});

app.use("/api/auth", authRoutes);

const PORT = process.env.PORT || 5001;

mongoose
  .connect(process.env.MONGODB_URI)
  .then(() => {
    console.log("MongoDB connected");
    app.listen(PORT, () => {
      console.log(`Server running on http://localhost:${PORT}`);
    });
  })
  .catch((error) => {
    console.error("MongoDB connection error:", error.message);
  });
```

middleware/authMiddleware.js

```
const jwt = require("jsonwebtoken");

function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Access token required" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = decoded.userId;
    next();
  } catch (error) {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
}

module.exports = authMiddleware;
```

models/User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
```

```

        type: String,
        required: true,
        trim: true
    },
    email: {
        type: String,
        required: true,
        unique: true,
        lowercase: true,
        trim: true
    },
    password: {
        type: String,
        required: true
    }
},
{ timestamps: true }
);

module.exports = mongoose.model("User", userSchema);

```

routes/authRoutes.js

```

const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");
const authMiddleware = require("../middleware/authMiddleware");

const router = express.Router();

// Register
router.post("/register", async (req, res) => {
    try {
        const { name, email, password } = req.body;

        if (!name || !email || !password) {
            return res.status(400).json({
                message: "Name, email and password are required"
            });
        }

        const existingUser = await User.findOne({ email });

        if (existingUser) {
            return res.status(409).json({ message: "Email already registered" });
        }

        const hashedPassword = await bcrypt.hash(password, 10);

        const user = await User.create({
            name,
            email,
            password: hashedPassword
        });

        res.status(201).json({
            message: "User registered successfully",
            user: {
                id: user._id,
                name: user.name,
                email: user.email
            }
        });
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});

// Login
router.post("/login", async (req, res) => {
    try {
        const { email, password } = req.body;

        const user = await User.findOne({ email });

        if (!user) {
            return res.status(401).json({ message: "Invalid email or password" });
        }

        const passwordMatch = await bcrypt.compare(password, user.password);

        if (!passwordMatch) {
            return res.status(401).json({ message: "Invalid email or password" });
        }

        const token = jwt.sign(
            { userId: user._id,
              process.env.JWT_SECRET,
              { expiresIn: "1h" }
            );

        res.json({
            message: "Login successful",
            token
        });
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});

// Protected profile route

```

```
router.get("/profile", authMiddleware, async (req, res) => {
  try {
    const user = await User.findById(req.userId).select("-password");

    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

    res.json(user);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

Assignment 3 — Notes App Backend

Purpose: Build a JWT-secured notes API with complete CRUD operations.

API endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login and get JWT
POST	/api/notes	Create note
GET	/api/notes	Get user's notes
GET	/api/notes/:id	Get one note
PUT	/api/notes/:id	Update note
DELETE	/api/notes/:id	Delete note

Postman testing example

First register/login using /api/auth endpoints on port 5002.

```
POST http://localhost:5002/api/notes
Authorization → Bearer Token
Body:
{
  "title": "My first note",
  "content": "Learn Express and MongoDB"
}
```

```
Expected:
{
  "_id": "NOTE_ID",
  "title": "My first note",
  "content": "Learn Express and MongoDB",
  "user": "USER_ID"
}
```

```
Then test:
GET /api/notes
PUT /api/notes/NOTE_ID
DELETE /api/notes/NOTE_ID
```

```
Without a valid JWT:
{
  "message": "Access token required"
}
```

Expected output / console flow

```
MongoDB connected
Server running on http://localhost:5002
```

```
POST /api/auth/register → 201 Created
POST /api/auth/login → 200 OK
POST /api/notes → 201 Created
GET /api/notes → 200 OK
PUT /api/notes/:id → 200 OK
DELETE /api/notes/:id → 200 OK
```

```
Missing JWT → 401 Unauthorized
```

Source code

.env.example

```
PORT=5002
MONGODB_URI=mongodb://127.0.0.1:27017/notes_db
JWT_SECRET=change_this_to_a_long_random_secret
```

README.md

```
# Assignment 3 - Notes App Backend
```

```
## Run
1. Copy `.env.example` to `.env`.
2. Set a private JWT_SECRET.
3. Run `npm install`.
4. Run `npm start`.
```

```
## Authentication
POST /api/auth/register
POST /api/auth/login
Use the JWT returned by login for all note requests:
```



```
Authorization: Bearer YOUR_TOKEN
```

```
## Notes CRUD
POST /api/notes
GET /api/notes
GET /api/notes/:id
PUT /api/notes/:id
DELETE /api/notes/:id

## Create Note JSON
{
  "title": "My first note",
  "content": "Learn Express and MongoDB"
}

## Expected create response
{
  "_id": "NOTE_ID",
  "title": "My first note",
  "content": "Learn Express and MongoDB",
  "user": "USER_ID"
}
```

All note queries are restricted to the authenticated user's ID.

package.json

```
{
  "name": "notes-app-backend",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "dotenv": "^16.4.5",
    "express": "^4.21.1",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.7.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.7"
  }
}
```

server.js

```
require("dotenv").config();

const express = require("express");
const mongoose = require("mongoose");
const authRoutes = require("./routes/authRoutes");
const noteRoutes = require("./routes/noteRoutes");

const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.json({ message: "Notes App Backend is running" });
});

app.use("/api/auth", authRoutes);
app.use("/api/notes", noteRoutes);

const PORT = process.env.PORT || 5002;

mongoose
  .connect(process.env.MONGODB_URI)
  .then(() => {
    console.log("MongoDB connected");
    app.listen(PORT, () => {
      console.log(`Server running on http://localhost:${PORT}`);
    });
  })
  .catch((error) => {
    console.error("MongoDB connection error:", error.message);
  });
```

middleware/authMiddleware.js

```
const jwt = require("jsonwebtoken");

function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Access token required" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = decoded.userId;
    next();
  } catch (error) {
    return res.status(401).json({ message: "Invalid token" });
  }
}
```

```

    return res.status(401).json({ message: "Invalid or expired token" });
  }
}

module.exports = authMiddleware;

```

models/Note.js

```

const mongoose = require("mongoose");

const noteSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: true,
      trim: true
    },
    content: {
      type: String,
      required: true
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model("Note", noteSchema);

```

models/User.js

```

const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    password: { type: String, required: true }
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);

```

routes/authRoutes.js

```

const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const router = express.Router();

router.post("/register", async (req, res) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: "All fields are required" });
    }

    if (await User.findOne({ email })) {
      return res.status(409).json({ message: "Email already registered" });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    const user = await User.create({
      name,
      email,
      password: hashedPassword
    });

    res.status(201).json({
      message: "Registration successful",
      user: { id: user._id, name: user.name, email: user.email }
    });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

router.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });

    if (!user || !(await bcrypt.compare(password, user.password))) {
      return res.status(401).json({ message: "Invalid email or password" });
    }

    const token = jwt.sign(
      { userId: user._id },
      process.env.JWT_SECRET,
      { expiresIn: "1h" }
    );
  }
});

```

```

    });

    res.json({ message: "Login successful", token });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;

```

routes/noteRoutes.js

```

const express = require("express");
const Note = require("../models/Note");
const authMiddleware = require("../middleware/authMiddleware");

const router = express.Router();

router.use(authMiddleware);

// Create note
router.post("/", async (req, res) => {
  try {
    const { title, content } = req.body;

    if (!title || !content) {
      return res.status(400).json({ message: "Title and content are required" });
    }

    const note = await Note.create({
      title,
      content,
      user: req.userId
    });

    res.status(201).json(note);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Get user's notes
router.get("/", async (req, res) => {
  try {
    const notes = await Note.find({ user: req.userId }).sort({ createdAt: -1 });
    res.json(notes);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Get one note
router.get("/:id", async (req, res) => {
  try {
    const note = await Note.findOne({
      _id: req.params.id,
      user: req.userId
    });

    if (!note) return res.status(404).json({ message: "Note not found" });

    res.json(note);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Update note
router.put("/:id", async (req, res) => {
  try {
    const note = await Note.findOneAndUpdate(
      { _id: req.params.id, user: req.userId },
      req.body,
      { new: true, runValidators: true }
    );

    if (!note) return res.status(404).json({ message: "Note not found" });

    res.json(note);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Delete note
router.delete("/:id", async (req, res) => {
  try {
    const note = await Note.findOneAndDelete({
      _id: req.params.id,
      user: req.userId
    });

    if (!note) return res.status(404).json({ message: "Note not found" });

    res.json({ message: "Note deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;

```

Learning Outcome

Students learn backend logic, database integration, REST API creation, authentication, password hashing, JWT-based route protection, and Postman API testing.

Important security note: use a private `JWT_SECRET` and never commit `.env` files or real credentials to GitHub.